The VLDB Journal manuscript No.
(will be inserted by the editor)

DBAutoDoc: Automated Discovery and Documentation of Undocumented Database Schemas via Statistical Analysis and Iterative LLM Refinement

Amith Nagarajan · Thomas Altman

Received: date / Accepted: date

Abstract The vast majority of production database systems in enterprise environments lack adequate documentation. Declared primary keys are absent, foreign key constraints have been dropped for performance, column names are cryptic abbreviations, and no entity-relationship diagrams exist. We present DBAutoDoc, a system that automates the discovery and documentation of undocumented relational database schemas by combining statistical data analysis with iterative large language model (LLM) refinement.

DBAutoDoc's central insight is that schema understanding is fundamentally an iterative, graph-structured problem. Drawing structural inspiration from backpropagation in neural networks, DBAutoDoc propagates semantic corrections through schema dependency graphs across multiple refinement iterations until descriptions converge. This propagation is discrete and semantic rather than mathematical, but the structural analogy is precise: early iterations produce rough descriptions akin to random initialization, and successive passes sharpen the global picture as context flows through the graph.

The system makes four concrete contributions: (1) an iterative context-propagation algorithm that refines table and column descriptions by re-analyzing each schema object in light of its neighbors' most recent descriptions; (2) a tiered statistical pipeline for primary key and foreign key discovery with a bidirectional feedback loop between LLM-generated semantic context and statistical candidate scoring; (3) a dual-layer human knowledge injection mechanism that distinguishes verified ground truth from exploratory seed context; and (4) a multi-criterion convergence detector that combines description stability windows, per-column confidence thresholds, and semantic change magnitude.

On a suite of benchmark databases, DBAutoDoc achieved overall weighted scores of 96.1% across two model families (Gemini and Anthropic) using a composite metric that weights key discovery accuracy (FK F1 at 35%, PK F1 at 30%) more heavily than description coverage (table descriptions at 20%, column descriptions at 15%). Ablation analysis demonstrates that the deterministic pipeline contributes a 23-point F1 improvement over LLM-only FK detection, confirming that the system's contribution is substantial and independent of LLM pre-training knowledge. The system reached convergence in 2 iterations on average at a cost of approximately $0.70 per 100 tables—a reduction of more than 99.5% relative to manual expert documentation. DBAutoDoc is released as open-source software with all evaluation configurations and prompt templates included for full reproducibility.

Keywords Database documentation · Schema discovery · Primary key detection · Foreign key detection · Large language models · Iterative refinement · Data profiling

1 Introduction

1.1 The Undocumented Database Problem

Relational databases are among the most durable artifacts in enterprise computing, yet they rarely preserve the knowledge of the humans who designed them. Column names like `cust_cd`, `trn_amt_3`, and `flg_b` encode

A. Nagarajan
Blue Cypress, New Orleans, LA, USA
E-mail: amith@bluecypress.io

T. Altman
Tasio Labs (a Blue Cypress company), Tuscaloosa, AL, USA
E-mail: thomas@tasio.co

meaning that once lived in the heads of developers who have long since moved on. Primary key declarations were dropped during bulk-load optimizations. Foreign key constraints were removed to improve insert throughput. ERD diagrams, when they existed at all, describe a schema from a decade before the current one.

We use the term *dark databases* to describe production systems with no declared keys, no column or table comments, no associated data dictionary, and no ERD documentation. Dark databases arise routinely in corporate acquisitions, data warehouse consolidation projects, and legacy migrations. While no large-scale empirical study has quantified the prevalence of undocumented schemas, data profiling research consistently identifies missing or incomplete metadata as a pervasive challenge in enterprise data management [1], and our own experience with enterprise clients (Sect. 7.5) confirms that completely undocumented production databases are common. For a database of moderate complexity—several hundred tables—manual documentation requires weeks of expert effort at a cost of $12,000–$48,000 in professional services.

1.2 Why Existing Approaches Fall Short

Several categories of existing tools address parts of the problem, but none addresses the full challenge of dark databases. *Schema extraction tools* (SchemaSpy, dbdocs.io) can only surface metadata that already exists. *One-shot LLM analysis* [34] cannot incorporate statistical evidence from actual data and treats each table in isolation, missing the iterative refinement that human experts naturally perform. *Schema matching tools* [27] identify candidate correspondences but do not produce human-readable documentation. *Data profiling frameworks* [1] generate statistical summaries without synthesizing them into natural-language documentation. *FK discovery algorithms* [28, 14] operate without semantic context and cannot distinguish coincidental numerical similarity from genuine referential relationships.

The core limitation shared by all these approaches is that they operate on a single pass over a static representation of the schema. Real schema understanding is an iterative process in which each piece of evidence revises prior assumptions and generates new hypotheses.

1.3 Our Insight: Schema Understanding as an Iterative Learning Problem

We observe that a database schema is not a collection of independent tables but a graph, where nodes are schema objects and edges represent referential relationships. Understanding a node depends on the state of neighboring nodes, which depend in turn on their neighbors. This graph-structured dependency motivates our iterative approach: an initial pass produces rough descriptions with high uncertainty, which are then used as context for successive passes that produce progressively better descriptions until convergence.

The structural analogy to backpropagation in neural network training [29] is instructive. In both systems, processing proceeds in a defined order, a loss or error signal is computed, correction information propagates backward through a dependency graph, and iteration continues until convergence. The analogy is structural rather than mathematical: there are no gradients, no differentiable loss function, and no parameter updates. The "corrections" are discrete revisions to natural-language descriptions, and "propagation" means providing updated neighbor descriptions as context for re-analysis. We are explicit that this is a design metaphor rather than a theoretical claim (see Sect. 8.2 for a detailed discussion of scope and limits).

1.4 Contributions

We summarize the contributions of this paper:

1. **Iterative context-propagation algorithm.** An algorithm that refines schema descriptions over multiple iterations by propagating semantic context through schema dependency graphs, the first to apply iterative graph-structured refinement to database documentation.
2. **Tiered statistical key discovery with LLM feedback.** A multi-stage pipeline combining cardinality analysis, value inclusion testing, naming heuristics, and deterministic gates with a bidirectional feedback loop between statistical discovery and LLM semantic validation.
3. **Dual-layer human knowledge injection.** A mechanism distinguishing ground truth injection (immutable constraints) from seed context injection (refinable priors), enabling effective collaboration between automated analysis and partial human expertise.
4. **Multi-criterion convergence detection.** A convergence detector combining description stability windows, per-column confidence thresholds, and semantic change magnitude to determine when further iteration yields diminishing returns.
5. **Cost analysis and open-source release.** A demonstration that DBAutoDoc reduces documentation cost by more than 99.5% relative to manual analysis,

with full release as open-source software including benchmark configurations, evaluation scripts, and prompt templates.

1.5 Paper Organization

Sect. 2 reviews related work. Sect. 3 provides a technical overview of the architecture. Sect. 4 describes the key discovery pipeline. Sect. 5 presents the iterative refinement algorithm. Sect. 6 covers implementation details. Sect. 7 reports experimental results including ablation analysis. Sect. 8 discusses limitations, threats to validity, and ethical considerations. Sect. 9 outlines future work. Sect. 10 concludes.

2 Related Work

DBAutoDoc sits at the intersection of several active research areas: schema understanding, LLM-based data interpretation, constraint discovery, iterative self-refinement, and human-in-the-loop learning.

2.1 Database Documentation and Schema Understanding

Traditional tools. SchemaSpy, ERAlchemy, and dbdocs.io extract and visualize existing metadata but produce empty output for dark databases. DBAutoDoc targets precisely the class where documentation must be inferred, not merely rendered.

Schema matching. [27] provide a comprehensive taxonomy of matching strategies. [16] offer systematic experimental comparison on realistic benchmarks. Schema matching assumes two schemas to be aligned; documentation must derive meaning from a single schema in isolation. DBAutoDoc borrows instance-level intuitions (value-overlap statistics) but operates in a single-schema setting.

LLM-based schema understanding. [34] address context-window overflow through succinct schema encoding. DBAutoDoc faces the same pressure but extends this to a *dynamic, iterative* context that evolves across refinement rounds. [18] combine a fine-tuned SLM for candidate generation with an LLM reranker. [30] take a self-improving approach philosophically similar to our iterative refinement, but their refinement targets pairwise column correspondences across schemas, while ours targets semantic coherence within a single schema's dependency graph.

Automated catalog metadata. [33] apply RAG to populating data catalog metadata, retrieving similar column descriptions from an existing catalog to guide generation for new columns. Their approach is fundamentally single-pass: each column is described independently with no cross-element context propagation, no statistical key discovery, and no iterative refinement. DBAutoDoc differs in three ways: (1) it operates on dark databases with no existing catalog to retrieve from; (2) it discovers relational structure (PKs/FKs) and uses it to order and contextualize analysis; and (3) descriptions are refined iteratively as context propagates through the schema dependency graph.

Semantic type detection. [12] frame semantic type detection as multi-class classification. DBAutoDoc incorporates similar instance-level features but treats semantic type detection as one input to a broader documentation task.

2.2 LLMs for Code and Data Understanding

Code documentation. Models such as Codex [6] and CodeBERT [9] generate documentation from source code. Database schemas occupy a middle ground between code and data, with formal structure and rich implicit semantics.

Text-to-SQL. Benchmarks including Spider [37] and BIRD [17] and systems like DAIL-SQL [10] and DIN-SQL [26] demonstrate that LLMs possess substantial knowledge about relational database conventions. These systems *consume* schema documentation; DBAutoDoc *produces* it.

Foundation models for data understanding. [20] demonstrate that large models generalize well to tabular data tasks with little fine-tuning—justifying DBAutoDoc's reliance on general-purpose LLMs. A common thread in this work is the single-pass evaluation paradigm; DBAutoDoc demonstrates that iterative refinement leaves substantial accuracy on the table.

2.3 Foreign Key and Primary Key Discovery

[28] formulate FK discovery as supervised classification over inclusion dependencies. [14] propose holistic PK/FK detection using joint reasoning. DBAutoDoc adopts a similar joint-reasoning philosophy in its bidirectional feedback loop. [15] and [13] address joinable column discovery and statistical correlation, respectively. FD discovery algorithms (TANE [11], FUN [22], HyFD [25]) provide signals for PK candidate assessment. IND detection algorithms (SPIDER [4], BINDER [24], S-INDD [7]) form the computational backbone of FK candidate generation [8]. The broader data profiling field [2,1,21] systematizes metadata extraction, with Metanome [23] providing a pluggable framework.

DBAutoDoc's principal contribution relative to this body of work is the *bidirectional feedback loop* between statistical discovery and LLM semantic validation, making them mutually reinforcing rather than sequential.

2.4 Iterative Refinement and Structured Reasoning

[19] introduce Self-Refine; [32] propose Reflexion with verbal reinforcement learning. Both demonstrate that iterative self-refinement substantially improves output quality. DBAutoDoc's refinement differs structurally: it is *graph-structured* rather than linear, with feedback coming from *structural neighbors* rather than self-critique. [3] propose Constitutional AI, where explicit principles govern revisions; DBAutoDoc's ground truth injections play an analogous anchoring role. [35] demonstrate Chain-of-Thought prompting; [36] extend this with Tree of Thoughts; and [5] generalize to Graph of Thoughts. DBAutoDoc's context propagation is a domain-specific instantiation of graph-structured reasoning, specialized for relational schema topology.

2.5 Human-in-the-Loop Machine Learning

Active learning [31] selects the most informative examples for human annotation. DBAutoDoc incorporates human input through ground truth injection (immutable anchors) and seed context (domain priors). Unlike classical active learning, ground truth annotations are immutable constraints rather than training signals, and their influence through the schema graph can be measured quantitatively.

3 System Architecture

Terminology. Throughout this paper, *primary key (PK) discovery* and *foreign key (FK) discovery* refer to inferring key constraints from data characteristics when no declared constraints exist. In the data profiling literature, FK discovery is formalized as *inclusion dependency (IND) detection* [24,8], and PK discovery relates to *unique column combination (UCC) detection* and *functional dependency (FD) discovery* [11,25]. We use the terms "PK/FK discovery" for readability, noting the formal equivalences where relevant.

3.1 Overview

DBAutoDoc is organized as a six-phase orchestrated pipeline, implemented in the `AnalysisOrchestrator` class. Each phase produces artifacts consumed by subsequent phases, and all intermediate state is persisted to disk for resumability. Figure 1 shows the component hierarchy.

```
AnalysisOrchestrator
|
+-- Phase 1: Schema Introspection
|   +-- DatabaseConnection
|   +-- Introspector
|
+-- Phase 2: Relationship Discovery
|   +-- DiscoveryEngine
|       +-- PrimaryKeyDetector
|       +-- ForeignKeyDetector
|
+-- Phase 3: Iterative Analysis
|   +-- AnalysisEngine
|       +-- DependencyLevelScheduler
|       +-- TableAnalyzer
|       +-- StatisticsCollector
|
+-- Phase 4: Sanity Checking
|   +-- LLMSanityChecker
|
+-- Phase 5: Sample Query Generation
|   +-- SampleQueryGenerator
|
+-- Phase 6: Output Generation
    +-- SQLGenerator
    +-- MarkdownGenerator
    +-- HTMLGenerator
    +-- CSVExporter
    +-- MermaidERDGenerator
```

Fig. 1 DBAutoDoc six-phase orchestration pipeline.

Phase 1 produces a raw schema graph; Phase 2 enriches it with inferred relationships; Phase 3 annotates every node with natural-language descriptions; Phase 4 applies cross-table consistency validation; Phase 5 synthesizes illustrative queries; Phase 6 serializes into multiple output formats.

3.2 Database Introspection Layer

The introspection layer constructs a complete, normalized schema representation through a driver abstraction: `BaseAutoDocDriver` defines a uniform interface implemented by concrete subclasses for SQL Server, PostgreSQL, and MySQL. For each table, the introspector retrieves column metadata (name, ordinal position, data type, nullability, defaults), existing descriptions, and constraint definitions. Row counts use platform-specific fast-count heuristics (e.g., `sys.dm_db_partition_stats` for SQL Server, `pg_class.reltuples` for PostgreSQL). All observed types are normalized to a canonical enumeration used uniformly by the discovery and analysis engines.

3.3 Data Sampling and Statistics Engine

For every column, the statistics engine computes cardinality, null fraction, min/max values, value frequency distribution, and type-specific profiles (percentiles for numerics, length distributions for strings, range summaries for dates). Default sample size is 1,000 rows per table using platform-specific random sampling. The resulting statistics are serialized into persistent state and attached to every subsequent LLM prompt, ensuring descriptions are grounded in actual observed distributions.

3.4 LLM Integration Layer

All prompts are defined as Nunjucks templates supporting conditionals, iteration, and macro composition. The pipeline uses 13 distinct prompt templates (see Appendix D). Every LLM invocation producing programmatic output specifies `responseFormat: 'JSON'` with a JSON Schema. All factual description tasks use temperature 0.1. The integration layer maintains separate input/output token counters per invocation, per phase, and per run, checked against guardrail limits before every call.

3.5 State Management and Resumability

All pipeline state is serialized to `state.json` incrementally after each phase boundary and dependency level. Each invocation receives a monotonically increasing run number. On restart, the orchestrator resumes from the earliest incomplete phase, skipping completed work at dependency-level granularity.

3.6 Output Formats

Phase 6 produces SQL annotation scripts (platform-specific DDL comments), Markdown documentation with table of contents, self-contained HTML reports, CSV exports, and Mermaid ERD diagrams.

3.7 Guardrails System

Three hard limits are configurable: `maxTokensPerRun`, `maxDurationSeconds`, and `maxCostDollars`. The token budget is partitioned across phases (default: 25% discovery, 70% analysis, 5% sanity checking). Warning thresholds at 80% enable graceful degradation. Pre-call enforcement ensures limits are checked synchronously before every LLM invocation.

4 Key Discovery Pipeline

4.1 Problem Formulation

Let $\mathcal{T} = \{T_1, T_2, \ldots, T_n\}$ be the tables in the target schema with columns $\mathcal{C}_i = \{c_{i,1}, c_{i,2}, \ldots, c_{i,m_i}\}$. The discovery engine must infer: (1) primary key candidates $\mathcal{P} = \{(T_i, K_i)\}$ where K_i uniquely identifies rows, and (2) foreign key candidates $\mathcal{F} = \{(T_i, c_{ij}, T_k, c_{kl})\}$ expressing referential relationships.

The design challenge is precision-recall balance. False positives waste LLM tokens and can actively mislead descriptions; false negatives cause related tables to be treated as independent, producing descriptions that omit important join paths.

4.2 Primary Key Detection

4.2.1 Candidate Generation and Hard Rejection

Candidates are generated through naming-pattern matching (e.g., `.*[Ii][Dd]$`) and uniqueness analysis ($u = |\text{distinct}(c)|/|\text{rows}(T)|$). Composite key detection tests two-column combinations when no single column achieves $u = 1.0$.

Hard rejection filters eliminate candidates that cannot be primary keys: columns with any nulls, columns where most values are empty strings or zero, and columns matching a semantic blacklist (date/time fields, quantity fields, financial fields, descriptive text fields).

4.2.2 Confidence Scoring

Surviving candidates receive a confidence score $s_{\text{PK}} \in [0, 100]$ via a weighted multi-factor model combining uniqueness (50%), naming pattern (20%), data type (15%), and data pattern (15%), with multiplicative penalties and a surrogate key boost. The default acceptance threshold is 70. Position-based heuristics (H9–H12) further improve precision by exploiting the strong convention that primary key columns occupy the first ordinal position. See Appendix A for detailed scoring formulas and Appendix B for the position heuristic specifications.

On AdventureWorks2022, position-based heuristics improved PK precision from 47.9% to 95.7% while recall decreased only marginally from 95.8% to 94.4%—a net F1 improvement from 48.0% to 95.0%.

4.3 Foreign Key Detection

4.3.1 Target-Finding Strategy

For each non-PK column, the engine identifies target columns through three strategies applied in sequence: (1) name-derived lookup (extracting candidate table names from column names like `CustomerID`), (2) PK naming similarity via Levenshtein distance, and (3) homonymous PK lookup as a fallback.

4.3.2 Tiered Pre-Filtering

A two-tier pre-filtering strategy eliminates the vast majority of candidates before value-level computation. Tier 1 excludes semantically incompatible types (DATE, BOOLEAN, FLOAT, BINARY, etc.) at zero cost. Tier 2 samples 10 values per candidate and applies pattern-based exclusion rules (emails, URLs, long strings) and promotion rules (UUIDs, numeric codes). This reduces candidates undergoing full containment analysis by 60–80%.

4.3.3 Multi-Factor Confidence Scoring and Deterministic Gates

FK candidates are scored by a weighted model combining value overlap (40%), naming similarity (20%), cardinality ratio (15%), target-is-PK bonus (15%), and null handling (10%). See Appendix A for complete formulas.

Before scoring, candidates pass through deterministic gates—hard, mathematically invariant filters that eliminate false positives with zero risk of rejecting correct relationships. These gates (G1–G8) eliminated approximately 75% of statistical false positives while losing zero correct FKs. A fan-out confidence penalty is applied when a single source column has candidates pointing to multiple targets. See Appendix A for gate specifications.

Adaptive Weight Redistribution. In schemas without declared PKs, the target-is-PK factor introduces systematic downward bias. When more than 40% of candidates score zero on this factor, the engine redistributes the PK bonus weight to value containment (increasing from 40% to 55%), preserving calibration.

4.3.4 The LLM as Primary FK Creator

A critical empirical finding is that the LLM, not the statistical pipeline, is the primary source of correct FK discoveries. In our AdventureWorks2022 evaluation:

- **Statistics-created FKs**: 15 correct out of 76 proposals (20% precision)
- **LLM-created FKs**: 75 correct out of 84 proposals (89% precision)

The LLM proposes FKs with high precision because it reasons about semantic relationships, while the statistical pipeline generates many false positives requiring filtering. Blocking LLM FK creation caused recall to drop from 90 to 49 correct FKs, confirming that LLM creative proposals are essential.

4.3.5 Bidirectional LLM Validation

Rather than a one-way pipeline, the system implements a feedback loop. In the statistical-to-LLM direction, candidates exceeding the confidence threshold undergo LLM semantic validation. In the LLM-to-statistical direction, the LLM proposes novel FK candidates during analysis that are then subjected to statistical value-containment checks before acceptance. This yields complementary coverage: statistics catch relationships with strong data evidence but non-obvious names; LLM inference catches relationships with clear semantic naming but imperfect data evidence.

5 Iterative Refinement via Context Propagation

5.1 Motivation and the Backpropagation Analogy

Automated schema documentation faces a bootstrapping problem analogous to training deep neural networks: the description of a parent table cannot be fully accurate without understanding its children, yet children depend on parent context. Consider an e-commerce schema: when the LLM first analyzes `Customers` in isolation, it may correctly infer account-level information. But analyzing `OrderItems` later—discovering per-item tax jurisdictions and fulfillment splits—reveals that `Orders` is a complex orchestration record, not merely a transaction header. This enrichment should flow back to improve `Orders` and potentially `Customers`.

The structural parallel to backpropagation is precise: (1) a forward pass processes components in dependency order; (2) an error signal is computed after forward processing; (3) correction information propagates backward through a dependency graph; (4) iteration repeats until convergence. The mechanism is entirely discrete and semantic—there are no gradients, no chain rule, no loss surfaces. This distinguishes our approach from single-artifact self-refinement [19,32] and connects to Graph of Thoughts [5], specialized for relational schema topology.

5.2 The Schema Dependency Graph

Let $\mathcal{S} = (T, C, R)$ denote a schema with tables T, columns C, and relationships $R \subseteq T \times T$. A topological ordering stratifies tables into dependency levels $L_0, L_1, \ldots, L_n$, where L_0 contains leaf tables (referenced by others but not themselves referencing any table). This ensures that when a table at level L_k is analyzed, all tables it references have already been processed and their descriptions are available as context.

Cross-schema foreign keys introduce inter-schema edges handled uniformly: the topological sort operates over all tables across all schemas. Cycles are broken conservatively by removing the lowest-confidence edge, though all edges are retained in LLM context.

5.3 Forward Pass: Initial Description Generation

The forward pass processes tables in dependency level order. For each table t at level L_k, the system assembles analysis context comprising: column statistics and sample values, related table descriptions from earlier levels, seed context, ground truth constraints, and (for iterations $i > 1$) prior iteration descriptions.

The LLM produces structured JSON output: a table description with confidence score $\sigma_t \in [0, 1]$, column descriptions with individual confidence scores, structured FK suggestions fed back to the discovery pipeline, and parent table insights—structured observations about parent tables that serve as upward-flowing correction signals (the semantic analogue of gradients).

5.4 Loss Computation: Sanity Checks

Sanity checks detect semantic inconsistencies at three granularities: dependency-level (after each level completes), schema-level (after each schema completes), and cross-schema (after all iterations). Six structural rules enforce PK/FK normalization principles. Violations generate structured error reports—the "loss signal"—queuing affected tables for re-analysis.

5.5 Context Propagation: The Backward Pass

Each iteration refines descriptions by incorporating updated neighbor context. Level 0 tables (no dependencies) are analyzed first, providing context for Level 1 tables. Insights from child tables propagate back to parent descriptions in subsequent iterations—the "backward pass" of our analogy.

The `BackpropagationEngine` implements insight accumulation, parent revision, and cascading propagation. After each level completes, insights from child tables are accumulated per parent. For each parent with non-empty insights and non-immutable description, a revision prompt presents the current description, accumulated insights with confidence scores, sanity-check violations, and seed context. The LLM returns `{needsRevision, revisedDescription, reasoning, confidence` allowing explicit determination that the current description is already correct.

Cascading propagation across iterations means a single pass propagates insights upward by exactly one dependency level; deeper corrections are realized in subsequent iterations. The number of iterations for full convergence is bounded by the dependency depth d. Ground truth tables are never revised, serving as fixed semantic anchors.

5.6 Convergence Detection

The `ConvergenceDetector` implements a multi-criterion stopping rule:

1. **Stability window**: No material changes in the last w iterations (default $w = 2$).
2. **Confidence threshold**: All individual confidence scores exceed minimum τ (default 0.6).
3. **Semantic comparison**: An LLM-based prompt distinguishes material from cosmetic changes.

Convergence is declared when at least two of three criteria are satisfied and at least two full iterations have completed. A hard maximum $K_{\max}$ (default 3) prevents unbounded execution.

5.7 Ground Truth and Seed Context

Ground truth entries are authoritative human-verified descriptions that function as immutable anchors—never overwritten by the LLM across any iteration. They propagate forward by inclusion in neighbors' context, analogous to labeled examples in semi-supervised learning.

Seed context provides domain guidance (overall purpose, business domains, industry context) injected into every prompt. It biases but does not constrain the LLM's interpretation, analogous to pre-trained weights in transfer learning.

Together, they dramatically reduce iterations required: schemas with both converge in 2 iterations; schemas with neither require 3–5.

5.8 Formal Algorithm

Algorithm 1 Iterative Schema Documentation with Context Propagation

Require: Schema $\mathcal{S} = (T, C, R)$; Ground Truth G; Seed Context SC; Config K
Ensure: Documentation $D : T \cup C \to \text{string}$

Introspect: Extract T, C from database catalog
Sample: Collect statistics and ≤ 10 values for each $c \in C$
Discover: Run PK/FK detection (Sect. 4) $\to$ augment R
for each $(t, \text{desc}) \in G$ **do**
  $D[t] \leftarrow$ desc; mark t as immutable
end for
$G_{\text{dep}} \leftarrow$ BuildDependencyGraph(T, R)
$L_0, L_1, \ldots, L_n \leftarrow$ TopologicalSort(G_{dep})
for iteration $i = 1$ to K.maxIterations **do**
  for each level $l = 0, 1, \ldots, n$ **do** ▷ Forward pass
    for each table $t \in L_l$ **do**
      if Immutable(t) **then continue**
      end if
      $\mathcal{C}(t) \leftarrow \{\text{stats}(t), \text{samples}(t), D[\text{parents}(t)], SC, D[\text{GT\_neighbors}(t)]\}$
      if $i > 1$ **then** $\mathcal{C}(t) \leftarrow \mathcal{C}(t) \cup \{D_{i-1}[t], \text{reasoning}_{i-1}[t]\}$
      end if
      $D'[t], \sigma_t, \mathit{\Pi}(t), F(t) \leftarrow$ LLM_Analyze$(t, \mathcal{C}(t))$
      FeedDiscovery$(F(t))$
    end for
    SanityCheck(L_l)
    for each parent p in $\bigcup_{t \in L_l} \text{parents}(t)$ **do** ▷ Propagation
      if Immutable(p) **then continue**
      end if
      $\hat{\mathit{\Pi}}(p) \leftarrow \bigcup_{t:(t,p) \in R,\, t \in L_l} \mathit{\Pi}(t)$
      if $\hat{\mathit{\Pi}}(p) \neq \emptyset$ **then**
        $\text{rev}, \sigma_p \leftarrow$ LLM_Revise$(p, D'[p], \hat{\mathit{\Pi}}(p))$
        if rev.needsRevision **then**
          $D'[p] \leftarrow$ rev.description
          Log$(p, \text{result} = \text{'changed'}, \text{iteration} = i)$
        else
          Log$(p, \text{result} = \text{'unchanged'}, \text{iteration} = i)$
        end if
      end if
    end for
  end for
  stable $\leftarrow$ NMaterialChanges(last $K.w$ iters) $= 0$
  confident $\leftarrow \forall\, t \in T : \sigma_t \geq K.\tau$
  semantic $\leftarrow \forall$ changed t : SemDiff$(D[t], D'[t]) =$ cosmetic
  if $|\{\text{stable}, \text{confident}, \text{semantic}\}| \geq 2$ **and** $i \geq 2$ **then**
    break
  end if
  $D \leftarrow D'$
end for
SanityCheck$(\mathcal{S})$; CrossSchemaSanityCheck$(\mathcal{S})$
return D'

The algorithm's complexity is $O(K_{\max} \cdot |T| \cdot \ell)$ LLM calls in the worst case. LLM calls within each dependency level are independent and issued in parallel, bounding wall-clock time by the depth n rather than $|T|$.

6 Implementation

6.1 Technology Stack

DBAutoDoc is implemented in TypeScript on Node.js, supporting SQL Server (`mssql`), PostgreSQL (`pg`), and MySQL (`mysql2`) through connection-pooled native drivers. LLM access is mediated through the MemberJunction AI abstraction layer, supporting Google Gemini, OpenAI GPT, Anthropic Claude, Groq, and Mistral. The tool is exposed as both a CLI (built on oclif) and a programmatic TypeScript API, available as the `@memberjunction/db-auto-do` package.

6.2 Prompt Engineering

The prompt layer comprises thirteen Nunjucks templates (see Appendix D). The central `table-analysis` template (∼140 lines) requests structured JSON containing table descriptions, confidence scores, column descriptions, FK suggestions, and parent table insights. The `backpropagation` template handles revision by presenting aggregated child observations. Sanity checking uses three scope-specific templates. A `semantic-comparison` template supports convergence detection.

All prompts use temperature 0.1. Context is scoped to direct ancestors and descendants in the FK graph rather than the entire schema. For models supporting effort-level parameters, high effort is used during sanity checks and lower effort during bulk analysis.

6.3 Token Budget Management

A three-tier guardrail system covers token count, wall-clock duration, and estimated cost. The global budget is partitioned across phases (25% discovery, 70% analysis, 5% sanity checking). Pre-call enforcement ensures limits are checked before every LLM invocation. Rate limiting uses exponential backoff with jitter.

6.4 Output Formats

Six formats are generated from a single run: SQL annotation scripts (platform-specific DDL), Markdown documentation, HTML reports, CSV exports, Mermaid ERD diagrams, and analysis-metrics reports. All outputs are organized into numbered run directories supporting side-by-side comparison.

7 Evaluation

7.1 Experimental Setup

7.1.1 Datasets

We evaluate DBAutoDoc on four public benchmark databases plus two private enterprise databases. Table 1 summarizes characteristics.

AdventureWorks2022 is the primary stress-test: 71 tables across five schemas with 91 declared FK relationships. For discovery experiments, all constraints and descriptions are stripped; originals serve as ground truth.

Chinook (11 tables, music store) provides clean FK relationships with unambiguous domain vocabulary for controlled evaluation.

Northwind (13 tables, order management) tests behavior on small schemas with limited statistical signal.

LousyDB is a purpose-built synthetic benchmark with intentionally cryptic abbreviated names (`cst`, `ord`, `prd`, `inv_ln`), no constraints, no descriptions, and intentional data quality issues—the archetype of a dark database not present in any LLM training data.

Full output artifacts (HTML documentation, ERD diagrams, SQL scripts, CSV exports) for all public benchmarks are available in the `results/` directory.

OrgA and **OrgB** are production enterprise databases (details in Sect. 7.5).

7.1.2 Metrics

Key discovery F1. Precision, recall, and F1 for PK and FK detection against ground truth (stripped constraint declarations).

Overall weighted score:

$$S_{\text{overall}} = 0.35 \cdot F1_{\text{FK}} + 0.30 \cdot F1_{\text{PK}} + 0.20 \cdot C_{\text{table}} + 0.15 \cdot C_{\text{col}} \quad (1)$$

where C_{table} is the fraction of tables receiving a non-empty description with confidence ≥ 0.5, and C_{col} is the fraction of columns receiving a non-empty description with confidence ≥ 0.5. Description coverage is a necessary-but-not-sufficient proxy for quality; it measures whether the system produced output, not whether the output is correct. We acknowledge this limitation: a formal description quality evaluation with human raters, structured Likert-scale rubrics, and inter-rater reliability measurement is planned as future work (Sect. 9, item 3). Key discovery (65% combined) is weighted more heavily than description coverage (35%) because structural relationships are objectively verifiable against ground truth and are prerequisites for meaningful documentation.

Convergence speed. Iterations until fewer than 5% of descriptions change materially.

Token efficiency and cost. Total tokens per table, broken down by phase, with USD cost at published provider pricing.

7.2 Key Discovery Results

7.2.1 Primary Key Detection

Northwind's lower score is partly attributable to evaluation artifacts (the `Order Details` table name with a space causes comparison mismatches) and low row counts in several tables limiting statistical signal.

7.2.2 Foreign Key Detection

7.2.3 Cross-Model Comparison

Key findings. (1) Context window matters: GPT-5.4-mini's 272K limit caused 15 FK misses vs. 1 for Gemini (1M) and 4 for Sonnet/Opus (680K). (2) The deterministic foundation provides a floor regardless of LLM choice. (3) Gemini and Anthropic achieve near-parity at 96.1%. (4) Sonnet/Opus used 7× fewer tokens than Gemini while achieving equivalent quality.

7.2.4 Cross-Database Summary

Token efficiency. Consumption scales sub-linearly for small databases (∼9K tokens/table for Chinook and Northwind) and increases for denser FK graphs (∼45K tokens/table for AdventureWorks). Total cost across all six databases was approximately $1–2 at Gemini Flash pricing.

7.3 Ablation Analysis

We frame the pipeline's development iterations as an ablation study, isolating the contribution of each architectural component to FK detection on AdventureWorks2022.

Key insight. The LLM contributes most of the *recall* (finding correct FKs), while the deterministic pipeline contributes most of the *precision* (removing false positives). Neither alone achieves the full pipeline's 94.2% F1. The deterministic gates account for a **23-point F1 improvement** over LLM-only detection (71.7% to 94.2%), demonstrating that the pipeline's contribution is substantial and independent of the LLM's pre-training knowledge.

Table 1 Benchmark Database Characteristics

Database	Tables	Columns	Domain	Constraints
AdventureWorks2022	71	486	Manufacturing/Sales	Full (stripped)
Chinook	11	64	Music store	Full (stripped)
Northwind	13	88	Order management	Full (stripped)
LousyDB	20	162	Synthetic (dark DB)	None by design
OrgA	36	1,807	Prof. membership assoc.	None (dark DB)
OrgB	125	2,347	Trade association	None (dark DB)

Table 2 Primary Key Detection Results

Database	Tables	True PKs	Detected	Precision	Recall	F1
AdventureWorks	71	71	70	95.7%	94.4%	95.0%
Chinook	11	11	11	95.2%	95.2%	95.2%
Northwind	13	13	11	72.7%	72.7%	72.7%

Table 3 Foreign Key Detection Results

Database	True	Det.	Prec.	Rec.	F1
AdventureWorks	91	100	90.0%	98.9%	94.2%
Chinook	11	11	95.2%	95.2%	95.2%
Northwind	13	12	75.0%	75.0%	75.0%

Table 4 Cross-Model Comparison on AdventureWorks2022

Metric	Gemini	GPT	Claude
PK F1	**95.0%**	89.4%	**95.0%**
FK F1	**94.2%**	77.9%	93.0%
Table Desc Cov.	99%	97%	100%
Column Desc Cov.	99%	96%	100%
Overall Score	**96.1% (A+)**	87.9% (B+)	**96.1% (A+)**
Tokens Used	3.2M	952K	471K

Table 5 Cross-Database Results (Gemini 3 Flash / 3.1 Pro, 2 Iterations)

Database	Tables	PK F1	FK F1	Table Desc	Col Desc	Weighted Score	Tokens
AdventureWorks	71	95.0%	94.2%	99%	99%	96.1% (A+)	3.2M
Chinook	11	95.2%	95.2%	100%	100%	96.9% (A+)	98K
Northwind	13	72.7%	75.0%	100%	100%	83.1% (B)	114K
LousyDB	20	97.6%	–*	100%	100%	–	214K

*LousyDB has no declared constraints by design. All 20 tables received correct PK identification (97.6% F1 accounting for scoring precision). FK ground truth requires manual extraction; a weighted score is not computable.

Table 6 FK Detection Ablation on AdventureWorks2022

Configuration	Correct	FK F1	Measures
Stats only (no LLM)	15/91	∼30%	Baseline
LLM only (no gates)	75/91	71.7%	LLM reasoning
Stats + LLM (no gates)	90/91	71.7%	Combined recall
+ Deterministic gates	87/91	87.0%	+Precision
Full pipeline	90/91	94.2%	+Pruning

For PK detection, the ablation is even starker: statistics alone found 34/71 PKs (48% F1), while the full pipeline with LLM PK proposals and position heuristics achieved 67/71 (95% F1)—a **47-point improvement**.

Interpretation. These results address a natural concern about training data contamination (see Sect. 8.4): if the LLM simply "knew" AdventureWorks relationships from pre-training, the deterministic pipeline would add no value. The 23-point F1 improvement from gates and pruning demonstrates that the pipeline's filtering and scoring mechanisms contribute substantial precision gains that the LLM cannot achieve alone. Furthermore, on private databases unseen by any LLM (OrgA, OrgB) and a purpose-built synthetic database (LousyDB), the system achieves comparable quality (Sect. 7.5), ruling out memorization as the primary explanation.

7.4 Iterative Refinement Results

Across all benchmark databases, DBAutoDoc converges within 2 iterations at median. The convergence rate exhibits a characteristic pattern: rapid change in iterations 1–2 as LLM descriptions incorporate FK-propagated context for the first time, followed by diminishing returns. We set a default maximum of 5 iterations, which captures the vast majority of achievable quality gain while bounding cost.

7.5 Enterprise Case Studies

7.5.1 OrgA: Professional Membership Association

Database profile. A professional association using a Salesforce-based CRM/AMS platform. The snapshot contained 36 tables, 1,807 columns across 4 schemas—a completely undocumented dark database with zero declared constraints.

Key discovery. DBAutoDoc detected 35/36 primary keys (97% coverage) and 193 foreign key relationships. The adaptive weight redistribution mechanism activated as expected.

Semantic highlights. The LLM correctly identified Salesforce Person Account patterns, the `__c` custom field suffix, the `NU` namespace prefix, committee governance structures, and membership lifecycle flows—all from cryptic API names without any human hints.

Estimated weighted score. While no ground truth exists for quantitative F1 measurement, we estimate 93–96% based on 97–100% PK coverage (35/36 correct, the remaining table having an ambiguous key structure), strong FK coverage validated by domain review, and 100% description coverage. The high PK coverage on this private database—which no LLM has seen in training—provides evidence that the system's key discovery generalizes beyond public benchmarks.

Processing. 2 iterations, 1.1M tokens, ∼1.5 hours, fully autonomous.

7.5.2 OrgB: Trade Association

Database profile. A custom .NET application with SQL Server backend: 125 tables, 2,347 columns across 10 schemas—completely undocumented.

Key discovery. 116/125 primary keys detected (93% coverage), 490 FK relationships (3.9 per table average). The 9 tables without PKs were primarily lookup tables with non-standard naming.

Semantic highlights. The system correctly identified the CRM schema's Customer table as the hub entity, discovered cross-schema relationships spanning Shopping, Purchasing, and Accounting schemas, and detected composite PKs in junction tables.

Estimated weighted score. We estimate 91–95% based on 93% PK coverage (116/125), strong cross-schema FK coverage validated by architectural review, and 100% description coverage. Like OrgA, this private database provides evidence of generalization independent of LLM pre-training.

Scaling comparison:

Cost scaled roughly linearly with table count, confirming economic viability on larger schemas.

7.6 Cost Analysis

The total LLM API cost across all six databases (276 tables, ∼5,100 columns) was approximately $1–2 at Gemini Flash pricing. Human documentation at $75–150/hr and 2–4 hours per table yields $15,000–60,000 for a 100-table database. DBAutoDoc achieves comparable documentation at under $1.00 per 100 tables—a reduction of more than 99.5%.

Token efficiency varies by model: Sonnet 4.6 / Opus 4.6 used only 471K tokens (7× fewer than Gemini's 3.2M) while achieving equivalent quality, making Anthropic models the most cost-effective option at current pricing.

7.7 Reproducibility

All benchmark results can be reproduced using the open-source release. The complete reproduction procedure is:

Table 7 Enterprise Scaling Comparison

Metric	OrgA (36 tbl)	OrgB (125 tbl)
PK coverage	97% (35/36)	93% (116/125)
FKs detected	193	490
Desc. coverage	100%	100%
Tokens consumed	1.1M	3.3M
Estimated cost	∼$0.20	∼$0.50

```
# 1. Install DBAutoDoc
npm install @memberjunction/db-auto-doc

# 2. Restore a benchmark database
#    (backups and stripping scripts
#     in research/v1/)

# 3. Create config.json pointing to
#    DB and LLM
#    (examples in research/v1/configs/)

# 4. Run analysis
db-auto-doc analyze --config ./config.json

# 5. Compare results against ground truth
python3 research/v1/scripts/compare.py \
  ./output/run-1/state.json
```

The following artifacts are included in the repository at `packages/DBAutoDoc/research/v1/`:

- Full output artifacts (HTML, Markdown, SQL, CSV, ERD, analysis reports) for all four public benchmarks across all three model families
- The `compare.py` evaluation script that computes PK/FK precision, recall, and F1 against ground truth constraint declarations
- All 13 prompt templates used during evaluation (version-controlled in `packages/DBAutoDoc/prompts/`)

Model identifiers and API versions are recorded in each run's `state.json` for traceability. Temperature 0.1 was used for all evaluation runs. Due to non-determinism in LLM inference, exact numerical reproduction may require the same model snapshot; directional results should be stable across model versions.

8 Discussion

8.1 When Context Propagation Helps Most

Propagation benefit scales with FK graph density: databases with many edges provide more propagation paths, and well-understood hub tables seed improvements across large neighborhoods. Schemas with consistent naming conventions converge faster. Hub tables benefit disproportionately from backpropagation, receiving revision prompts incorporating use-case evidence from all consumers. Diminishing returns emerge for large, loosely connected schemas where the graph partitions into weakly linked components.

8.2 The Backpropagation Analogy: Scope and Limits

The term "backpropagation" is used deliberately but with acknowledged looseness. The structural parallels are genuine: defined processing order, backward-flowing correction signals, iterative convergence. The differences are equally genuine: there are no continuous gradients, no weight updates, no differentiable objective. We do not claim mathematical equivalence. The contribution is heuristic: an intuitive mental model that makes clear the approach is categorically different from single-pass summarization.

8.3 Limitations

Several limitations bound our claims. (1) LLM hallucination remains an irreducible risk; generated documentation should be reviewed by domain experts for safety-critical processes. (2) Statistical FK discovery is bounded by pattern recognizability; unconventional key designs may exhibit lower recall. (3) Token cost scales with database size; very large databases incur non-trivial charges. (4) Quality degrades for empty or near-empty tables where sample-value statistics are unavailable. (5) Non-English column names reduce LLM effectiveness.

8.4 Threats to Validity

Training data contamination. AdventureWorks, Northwind, and Chinook are widely known databases likely present in LLM training data. The LLM may "know" PK/FK relationships from pre-training rather than discovering them through our pipeline. We offer three mitigations: (1) OrgA and OrgB are private databases no

LLM has seen, achieving 97% and 93% PK coverage respectively with estimated weighted scores of 91–96%; (2) LousyDB was purpose-built for this evaluation and is not in any training data, achieving 97.6% PK F1; (3) ablation analysis (Sect. 7.3) shows the deterministic pipeline contributes a 23-point F1 improvement beyond what the LLM achieves alone, demonstrating substantial value independent of memorization. Future work includes additional synthetic databases designed to further control for this threat.

Limited database diversity. Results span 4 public and 2 private databases. While these cover multiple domains (manufacturing, music, trade associations, education), more diverse schemas—NoSQL exports, data warehouses, time-series databases, and schemas with non-Western naming conventions—would strengthen generalization claims.

Single evaluator for descriptions. Ground truth comparison for key discovery is automated against declared constraints, but description quality has not been evaluated by independent human raters in this version of the paper. We plan a formal human evaluation study with multiple domain-expert raters and inter-rater reliability measurement.

SQL Server focus. All benchmarks use SQL Server. PostgreSQL and MySQL drivers exist and are tested for connectivity and metadata extraction, but have not been included in the quantitative benchmark suite.

LLM model version sensitivity. Provider model updates can alter output characteristics without version-number changes. We record model identifiers and API versions for each result, but exact replication may require the same model snapshot. Temperature 0.1 reduces but does not eliminate variance.

Domain shift and naming conventions. All benchmark databases use English column names and Western naming conventions (e.g., `CustomerID`, `OrderDate`). Databases with non-English names, domain-specific abbreviations (e.g., medical coding systems, financial instrument identifiers), or unconventional naming patterns may exhibit lower discovery recall and description quality. The LousyDB benchmark with intentionally cryptic names (e.g., `cst`, `ord`, `inv_ln`) partially addresses this threat but does not cover non-Latin scripts or domain-specific vocabularies.

Model context window and pricing claims. Context window sizes and per-token pricing cited in this paper reflect provider documentation as of Q1 2026. These values are subject to change without notice as providers update models and pricing. We cite specific model identifiers (e.g., "Gemini 3 Flash," "Claude Sonnet 4.6") rather than generic model families to improve traceability.

8.5 Ethical Considerations and Data Privacy

The primary ethical concern is data privacy. DBAutoDoc transmits the following information to LLM providers in each prompt: (1) table and column names, (2) data types and nullability constraints, (3) statistical summaries (cardinality, null fraction, min/max values, value frequency distributions), and (4) up to 10 sampled row values per column. Items (3) and (4) may contain personally identifiable information (PII), financial data, or other sensitive values.

DBAutoDoc provides several configurable controls to mitigate this risk. The `sampleSize` configuration parameter can be set to zero, which disables all value sampling and value-frequency analysis—reducing prompts to structural metadata only (items 1–2 above), at the cost of reduced description quality. The `cardinalityThreshold` parameter controls which columns receive detailed statistical profiling. Schema and table filters (`schemas.include`, `schemas.exclude`, `tables.exclude`) allow sensitive tables to be excluded entirely from analysis.

For organizations subject to GDPR, HIPAA, or similar regulations, we recommend: (a) setting `sampleSize: 0` for databases containing PII, (b) using schema/table exclusion filters to skip sensitive tables, or (c) deploying locally-hosted LLM models to keep all data on-premises. DBAutoDoc's LLM integration layer supports any provider accessible through MemberJunction's AI abstraction, including local inference servers. Users bear responsibility for evaluating regulatory compliance for their specific deployment context.

9 Future Work

Based on our benchmark experience, we identify six concrete directions:

1. **Cross-platform validation.** Extending the quantitative benchmark suite to PostgreSQL and MySQL to validate driver parity and confirm that key discovery heuristics transfer across platforms.
2. **Parallelization for large databases.** Implementing concurrent table analysis within dependency levels and parallel schema processing to reduce wall-clock time for enterprise-scale schemas (500+ tables).
3. **Description quality human evaluation.** Conducting a formal study with multiple independent domain-expert raters, structured Likert-scale rubrics, and inter-rater reliability measurement using Cohen's kappa.
4. **Training data contamination mitigation.** Creating additional synthetic test databases (following the LousyDB model) with diverse naming conventions,

key structures, and domain vocabularies to provide contamination-free benchmarks.

5. **Cost optimization for large schemas.** Implementing confidence-driven model dispatch (routing high-confidence tables to lightweight models, escalating low-confidence tables to stronger models) and context-window optimization to reduce token consumption by an estimated 40–60%.
6. **Driver refactoring with SQLDialect integration.** Leveraging MemberJunction's existing SQLDialect abstraction to eliminate platform-specific SQL throughout the introspection and statistics layers.

Additional longer-term directions include active learning for ground truth selection, cross-database enterprise-scale analysis, incremental/continuous analysis integrated with CI/CD pipelines, local and hybrid LLM deployment for data sovereignty, and ecosystem integration with data catalog platforms and IDE plugins.

10 Conclusion

Undocumented databases represent one of the most persistent and costly problems in enterprise software engineering. Comprehensive manual documentation requires weeks of expert effort and tens of thousands of dollars—a cost most organizations cannot justify.

DBAutoDoc addresses this through a bidirectional feedback loop between statistical key discovery and iterative LLM-based analysis organized along the discovered FK graph. Statistical discovery recovers relational structure; context propagation, inspired structurally by backpropagation, refines descriptions across multiple passes until semantic convergence. Parent tables receive insights from their children; children receive context from their now-better-understood parents.

Empirical evaluation demonstrates 95.0% F1 on primary key detection and 94.2% F1 on foreign key detection with 99% description coverage on AdventureWorks2022. Ablation analysis confirms that the deterministic pipeline contributes a 23-point F1 improvement independent of LLM pre-training knowledge. On private enterprise databases unseen by any LLM, the system achieves 93–97% PK coverage with estimated weighted scores of 91–96%, providing evidence that results generalize beyond public benchmarks. Convergence is typically achieved within 2 iterations at a cost under $1 per 100 tables—a reduction of more than 99.5% relative to manual documentation.

The central finding is that *treating schema documentation as an iterative learning problem* rather than a one-shot extraction task produces substantially better results. Neither the statistical approach alone nor the LLM approach alone achieves the quality of their combination in a bidirectional feedback loop.

DBAutoDoc is released as open-source software under the MIT License as part of the MemberJunction platform, with all benchmark configurations, prompt templates, and evaluation scripts included for full reproducibility.[1]

A Deterministic Gate and Scoring Specifications

A.1 PK Confidence Scoring Formula

Candidates surviving hard rejection filters are scored via:

$$s_{\text{PK}} = 50 \cdot f(u) + 20 \cdot n + 15 \cdot d + 15 \cdot p \tag{2}$$

Uniqueness factor $f(u)$ (50% weight): $f(u) = u$ when $u \geq 0.95$; linearly scaled below.

Naming factor n (20% weight): 1.0 if column name matches a PK naming pattern; 0 otherwise.

Data type factor d (15% weight): INT/BIGINT/UNIQUEIDENTIFIER → 1.0; VARCHAR → 0.6; TEXT/BLOB → 0.2; other → 0.3.

Data pattern factor p (15% weight): Sequential integers → +15; GUID/UUID → +15; natural key codes → +10; composite-derived → +5.

Penalties. Nulls: ×0.7. Unique but atypical name: ×0.5. FK likelihood ℓ_{FK}: $\times(1 - 0.6 \cdot \ell_{\text{FK}})$.

Surrogate Key Boost. +20 points when: name matches `id` or {`table_name`}`_id`, $u \geq 0.95$, and $d \geq 0.9$.

Default acceptance threshold: 70.

A.2 FK Confidence Scoring Formula

$$s_{\text{FK}} = 40 \cdot v + 20 \cdot s + 15 \cdot r + 15 \cdot k + 10 \cdot \nu \tag{3}$$

Value overlap v (40%): Fraction of sampled source values (up to 500) appearing in target column.

Naming similarity s (20%): Normalized Levenshtein distance. Full match → 1.0; containment → 0.8; partial matches scaled linearly.

Cardinality ratio r (15%): $r = \min(\rho, 2)/2$ where $\rho = |\text{rows}(T_i)|/|\text{distinct}(c_{ij})|$.

Target-is-PK bonus k (15%): 1.0 if target is a detected PK; 0 otherwise.

Null handling ν (10%): Null fraction <30% → 1.0; 30–70% → 0.5; >70% → 0.

Penalties. Orphan rate >20%: ×0.7. Incompatible types: ×0.5. Default threshold: 60.

A.3 Deterministic Gates (G1–G8)

Gate G1: Target PK-Eligibility. Target column must pass PK hard rejection filters (uniqueness, non-null, non-blank). Logically irrefutable: FKs reference PKs by definition.

[1] Repository: `https://github.com/MemberJunction/MJ` — DBAutoDoc package: `https://github.com/MemberJunction/MJ/tree/next/packages/DBAutoDoc`

Gate G3: Rowguid Target Filter. Columns named `rowguid` are excluded as FK targets. These are system-generated replication identifiers, never semantically meaningful FK targets.

Gate G4: Row-Count Ratio Confidence Multiplier. Scales confidence downward when source table has dramatically fewer rows than target, violating the expected many-to-one FK cardinality pattern.

Gate G5: Fan-Out Limiter (Top-3). When a source column has candidates pointing to multiple targets, only the top 3 by confidence are retained. No correct FK was ranked 4th or lower in our evaluation.

Gate G6: Value Overlap Threshold (75%). Candidates with <75% source value containment are rejected. Calibrated to accommodate data quality issues while eliminating coincidental overlaps.

Gate G8: Source-is-PK Skip. PK columns are skipped as FK sources (except self-referencing hierarchies handled separately). Eliminates false positives from coincidental PK-to-PK value overlap.

A.4 Fan-Out Confidence Penalty

$$\psi(n) = \begin{cases} 1.0 & \text{if } n = 1 \\ 0.85 & \text{if } n = 2 \\ 0.75 & \text{if } n = 3 \\ 0.65 & \text{if } n \geq 4 \end{cases} \tag{4}$$

where n is the number of target tables after gate filtering. Pushes ambiguous candidates below the confidence locking threshold, delegating final arbitration to LLM-based pruning.

B PK Position Heuristics (H9–H12)

Heuristic H9: Column Position Scoring. The confidence score is multiplied by:

$$\phi(\text{pos}) = \begin{cases} 1.0 & \text{if pos} = 0 \\ 0.85 & \text{if pos} = 1 \\ 0.70 & \text{if pos} = 2 \\ 0.55 & \text{if pos} \geq 3 \end{cases} \tag{5}$$

where pos is the column's zero-indexed ordinal position. The aggressive discount for pos ≥ 3 reflects that such columns are rarely primary keys. In AdventureWorks2022, every correct PK began at position 0.

Heuristic H10: Consecutive Composite Key Detection. Composite key candidates whose constituent columns form a contiguous prefix of the table's column list (positions 0, 1, 2, ...) receive a confidence boost, reflecting the convention that composite keys occupy leading columns.

Heuristic H11: Progressive Discount for Later PK-Eligible Columns. When multiple columns pass hard filters and achieve high uniqueness, progressive multiplicative discounts are applied beyond the first eligible column. Tables rarely have multiple independent surrogate keys.

Heuristic H12: Composite Supersedes Individual. When a composite key achieves high confidence and its constituent columns are individually PK-eligible, individual candidates are suppressed. Prevents reporting both individual columns and their composite as separate PKs.

C Per-Run Benchmark History

D Prompt Templates

DBAutoDoc uses 13 prompt templates, all version-controlled in the open-source repository. Table 9 summarizes the key templates.

All templates are available in the DBAutoDoc prompts directory on GitHub.

Table 8 Key Milestone Runs (AdventureWorks2022)

Run	FK	FK F1	PK F1	Notes
005	90/91	71.7%	∼48%	No gates
008	87/91	82.4%	89.0%	+G1, G3, G5
011	87/91	87.0%	95.0%	+Position H9–H12
015	90/91	94.2%	95.0%	Full pipeline

Table 9 Prompt Template Summary

Template	Scope	Purpose
table-analysis	Per-table	Descriptions, FK suggestions, insights
backpropagation	Per-parent	Revision from child insights
fk-pruning	Per-column	FK candidate arbitration
pk-pruning	Per-table	PK semantic validation
dep-level-sanity	Per-level	Cross-table consistency
schema-sanity	Per-schema	Naming consistency
cross-schema-sanity	Global	Entity alignment
semantic-comparison	Per-table	Change classification
query-planning	Per-table	Use-case identification
query-generation	Per-query	SQL synthesis
query-fix	Per-query	SQL correction
query-refinement	Per-query	Result adjustment

References

1. Abedjan, Z., Golab, L., Naumann, F.: Profiling relational data: A survey. The VLDB Journal **24**(4), 557–581 (2015). DOI 10.1007/s00778-015-0389-y
2. Abedjan, Z., Golab, L., Naumann, F., Papenbrock, T.: Data Profiling. Synthesis Lectures on Data Management. Morgan & Claypool (2018). DOI 10.2200/S00878ED1V01Y201810DTM052
3. Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al.: Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073 (2022)
4. Bauckmann, J., Leser, U., Naumann, F., Tietz, V.: Efficiently detecting inclusion dependencies. In: Proceedings of the IEEE International Conference on Data Engineering (ICDE), pp. 1448–1450 (2007)
5. Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Nyczyk, H., Slowinski, P., Hoefler, T.: Graph of thoughts: Solving elaborate problems with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 17,682–17,690 (2024). DOI 10.1609/aaai.v38i16.29720
6. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
7. De Marchi, F., Lopes, S., Petit, J.M.: Unary and n-ary inclusion dependency discovery in relational databases. Journal of Intelligent Information Systems **32**(1), 53–73 (2009)
8. Dürsch, F., Stebner, A., Windheuser, F., Fischer, M., Friedrich, T., Streubel, N., Bleifuß, T., Harmouch, H., Jiang, L., Papenbrock, T., Naumann, F.: Inclusion dependency discovery: An experimental evaluation of thirteen algorithms. In: Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM), pp. 219–228 (2019). DOI 10.1145/3357384.3357916
9. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M.: CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547 (2020). DOI 10.18653/v1/2020.findings-emnlp.139
10. Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., Zhou, J.: Text-to-SQL empowered by large language models: A benchmark evaluation. Proceedings of the VLDB Endowment **17**(5), 1132–1145 (2024). DOI 10.14778/3641204.3641221
11. Huhtala, Y., Kärkkäinen, J., Porkka, P., Toivonen, H.: TANE: An efficient algorithm for discovering functional and approximate dependencies. The Computer Journal **42**(2), 100–111 (1999). DOI 10.1093/comjnl/42.2.100
12. Hulsebos, M., Hu, K., Bakker, M., Zgraggen, E., Satyanarayan, A., Kraska, T., Demiralp, c., Hidalgo, C.: Sherlock: A deep learning approach to semantic data type detection. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 1500–1508 (2019). DOI 10.1145/3292500.3330993
13. Ilyas, I.F., Markl, V., Haas, P., Brown, P., Aboulnaga, A.: CORDS: Automatic discovery of correlations and soft functional dependencies. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 647–658 (2004). DOI 10.1145/1007568.1007641
14. Jiang, L., Naumann, F.: Holistic primary key and foreign key detection. Journal of Intelligent Information Systems **54**(3), 439–461 (2020). DOI 10.1007/s10844-019-00562-z
15. Khatiwada, A., Shraga, R., Gatterbauer, W., Miller, R.J.: Integrating data lake tables. Proceedings of the VLDB Endowment **16**(4), 932–945 (2022). DOI 10.14778/3574245.3574274
16. Koutras, C., Siachamis, G., Ionescu, A., Psarakis, K., Brons, J., Bozovic, M., Karagiannis, S., van Keulen, M., Stoyanovich, J., Abedjan, Z.: Valentine: Evaluating matching techniques for dataset discovery. In: Proceedings of the IEEE International Conference on Data Engineering (ICDE), pp. 468–479 (2021). DOI 10.1109/ICDE51399.2021.00047
17. Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., et al.: Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
18. Liu, Y., Pena, E.H.M., Santos, A., Wu, E., Freire, J.: Magneto: Combining small and large language models for effective schema matching. Proceedings of the VLDB Endowment **18**(8), 2681–2694 (2025)
19. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al.: Self-refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
20. Narayan, A., Chami, I., Orr, L., Ré, C.: Can foundation models wrangle your data? Proceedings of the VLDB Endowment **16**(4), 738–746 (2022). DOI 10.14778/3574245.3574258
21. Naumann, F.: Data profiling revisited. ACM SIGMOD Record **42**(4), 40–49 (2013). DOI 10.1145/2590989.2590995
22. Novelli, N., Cicchetti, R.: FUN: An efficient algorithm for mining functional and embedded dependencies. In:

Proceedings of the International Conference on Database Theory (ICDT), *LNCS*, vol. 1973, pp. 189–203. Springer (2001). DOI 10.1007/3-540-44503-X_13
23. Papenbrock, T., Ehrlich, J., Marten, J., Neuber, T., Rudolph, J.P., Schönberg, M., Zwiener, J., Naumann, F.: Data profiling with Metanome. Proceedings of the VLDB Endowment **8**(12), 1860–1863 (2015). DOI 10.14778/2824032.2824086
24. Papenbrock, T., Ehrlich, J., Marten, J., Neuber, T., Rudolph, J.P., Schönberg, M., Zwiener, J., Naumann, F.: Divide & conquer-based inclusion dependency discovery. Proceedings of the VLDB Endowment **8**(7), 774–785 (2015). DOI 10.14778/2752939.2752946
25. Papenbrock, T., Naumann, F.: A hybrid approach to functional dependency discovery. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp. 821–833 (2016). DOI 10.1145/2882903.2915203
26. Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
27. Rahm, E., Bernstein, P.A.: A survey of approaches to automatic schema matching. The VLDB Journal **10**(4), 334–350 (2001). DOI 10.1007/s007780100057
28. Rostin, A., Albrecht, O., Bauckmann, J., Naumann, F., Stillger, M.: A machine learning approach to foreign key discovery. In: Proceedings of the International Workshop on the Web and Databases (WebDB), SIGMOD Workshop (2009)
29. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**, 533–536 (1986)
30. Seedat, N., van der Schaar, M.: Matchmaker: Self-improving large language model programs for schema matching. In: NeurIPS 2024 Workshop on Table Representation Learning (2024)
31. Settles, B.: Active learning literature survey. Tech. Rep. CS Tech Report 1648, University of Wisconsin–Madison (2009)
32. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
33. Singh, M., Kumar, A., Donaparthi, S., Karambelkar, G.: Leveraging retrieval augmented generative LLMs for automated metadata description generation to enhance data catalogs. arXiv preprint arXiv:2503.09003 (2025)
34. Trummer, I.: Generating succinct descriptions of database schemata for cost-efficient prompting of large language models. Proceedings of the VLDB Endowment **17**(11), 3511–3523 (2024). DOI 10.14778/3681954.3682017
35. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2022)
36. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
37. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3911–3921 (2018). DOI 10.18653/v1/D18-1425